

```

// Log.h
#ifndef LOG_H
#define LOG_H

#include <iostream>
#include <string>

// LogLevel is a small helper class used by level("warning"),
// level("error") and level("fatal").
class LogLevel {
public:
    explicit LogLevel(const std::string& name);

private:
    int m_value;

    friend class Log;
};

// Helper function used like: obj << level("fatal") << "BROKEN" << endl;
LogLevel level(const std::string& name);

class Log {
public:
    Log();

    // Set the global lowest output level. Messages whose level is lower
    // than this value will not be printed.
    void set_level(const std::string& name);

    // Receive a message level from level("...").
    Log& operator<<(const LogLevel& lv);

    // Generic stream output operator. It allows usages such as:
    // obj << "message" << 123 << std::endl;
    template <typename T>
    Log& operator<<(const T& value) {
        if (m_enabled) {
            std::cout << value;
        }
        return *this;
    }

    // Support manipulators such as std::endl, std::flush and std::ends.
    Log& operator<<(std::ostream& (*manip)(std::ostream&));

    // Support manipulators such as std::hex, std::dec and std::boolalpha.
    Log& operator<<(std::ios& (*manip)(std::ios&));
    Log& operator<<(std::ios_base& (*manip)(std::ios_base&));

private:
    enum Severity {
        WARNING = 1,
        ERROR = 2,
        FATAL = 3
    };

    static int s_globalLevel; // shared by all Log objects
    int m_messageLevel;      // level of the current message
    bool m_enabled;          // whether the current message can be printed

    friend class LogLevel;

    static int stringToLevel(const std::string& name);
    void updateEnabled();
    void resetMessageLevel();
};

#endif

```

```

// Log.cpp
#include "Log.h"

#include <algorithm>
#include <cctype>

int Log::s_globalLevel = Log::WARNING;

LogLevel::LogLevel(const std::string& name)
    : m_value(Log::stringToLevel(name)) {
}

LogLevel level(const std::string& name) {
    return LogLevel(name);
}

Log::Log()
    : m_messageLevel(WARNING), m_enabled(true) {
    updateEnabled();
}

void Log::set_level(const std::string& name) {
    s_globalLevel = stringToLevel(name);
    updateEnabled();
}

Log& Log::operator<<(const LogLevel& lv) {
    m_messageLevel = lv.m_value;
    updateEnabled();
    return *this;
}

Log& Log::operator<<(std::ostream& (*manip)(std::ostream&)) {
    if (m_enabled) {
        manip(std::cout);
    }

    // In this simple logger, an ostream manipulator such as std::endl
    // is treated as the end of one log message.
    resetMessageLevel();
    return *this;
}

Log& Log::operator<<(std::ios& (*manip)(std::ios&)) {
    if (m_enabled) {
        manip(std::cout);
    }
    return *this;
}

Log& Log::operator<<(std::ios_base& (*manip)(std::ios_base&)) {
    if (m_enabled) {
        manip(std::cout);
    }
    return *this;
}

int Log::stringToLevel(const std::string& name) {
    std::string s = name;
    std::transform(s.begin(), s.end(), s.begin(),
        [](unsigned char ch) { return static_cast<char>(std::tolower(ch)); });

    if (s == "fatal") {
        return FATAL;
    }
    if (s == "error") {
        return ERROR;
    }

    // Unknown levels are treated as warning to keep the class easy to use.
    return WARNING;
}

```

```
void Log::updateEnabled() {
    m_enabled = (m_messageLevel >= s_globalLevel);
}

void Log::resetMessageLevel() {
    m_messageLevel = WARNING;
    updateEnabled();
}

// main.cpp
#include "Log.h"

#include <iostream>

using namespace std;

int main() {
    Log obj;

    // Basic stream-like output.
    obj << "debug message" << endl;

    // Only messages whose level is error or fatal can be printed now.
    obj.set_level("error");

    // warning is lower than error, so this line will not be printed.
    obj << level("warning") << "WARNING MESSAGE" << endl;

    // fatal is higher than error, so this line will be printed.
    obj << level("fatal") << "BROKEN" << endl;

    return 0;
}
```